Construindo uma cultura de code review 🚀

Renan de Assis



Sumário

1. O que são PRs e Code Reviews?
2. Benefícios do Code Review
3. Code Review na era da IA
4. Dicas para autores de PRs
5. Dicas para revisores de PRs

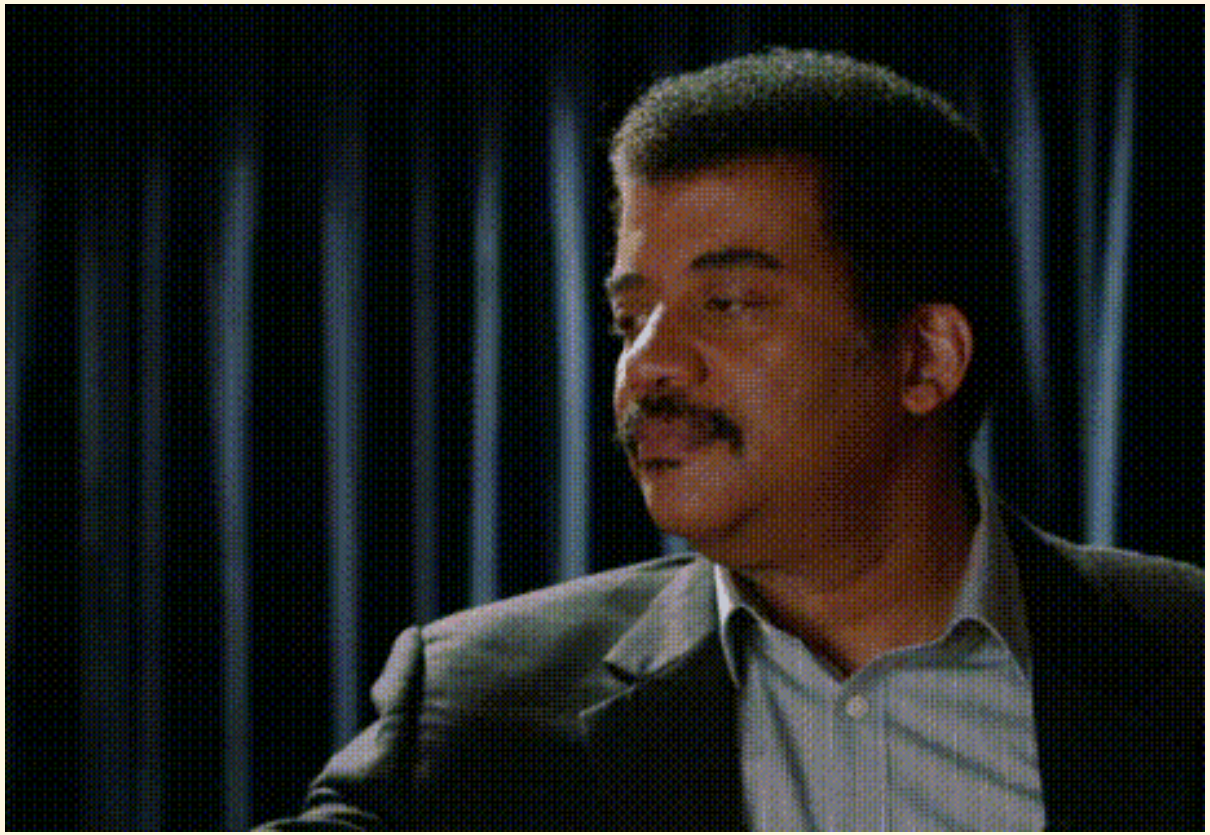
Quem sou eu?

- (auto descrição)



Quem sou eu?

- Bacharel em Física





Quem sou eu?

- Bacharel em Física
- Engenheiro de software na Serasa





Quem sou eu?

- Jogo vôlei
- Adoro jogos de tabuleiro e tenho uma tatuagem do desenho Avatar
- Ajudei uns anos na organização dos eventos Python Brasil

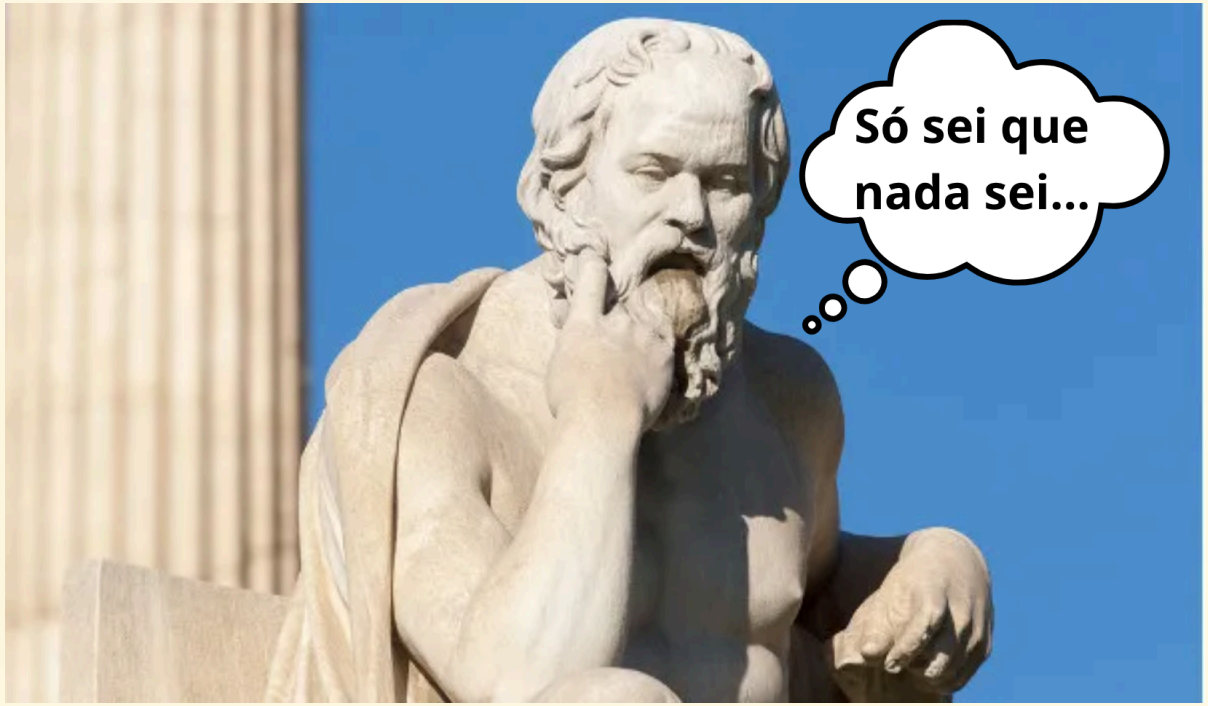




AVISOS

AVISOS

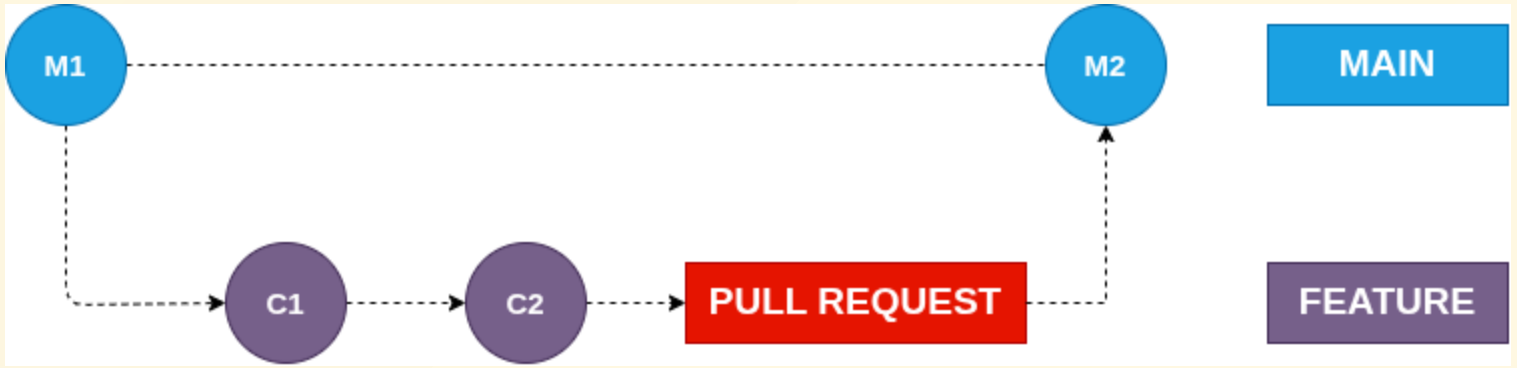
- Minhas considerações do que **eu** aprendi na vida
- Tudo aqui é debatível



O que são PRs e Code Reviews?

O que são PRs?

“ “ Pull Request é um recurso de plataformas de hospedagem de código Git para propor mudanças de uma branch para outra em um projeto de software.[1] ” ”



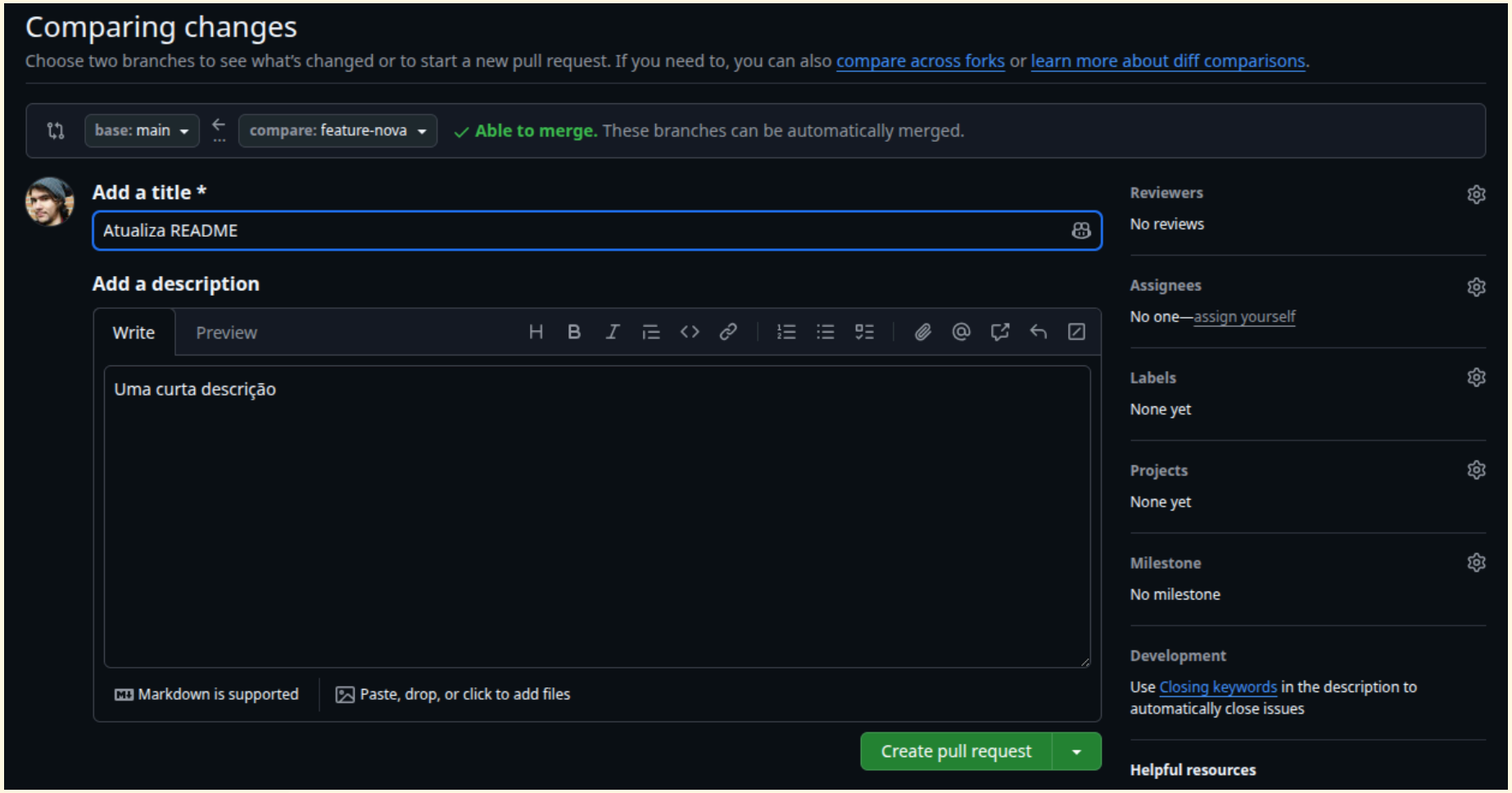
1. Definição própria

O que são ~~PRs~~ MRs?

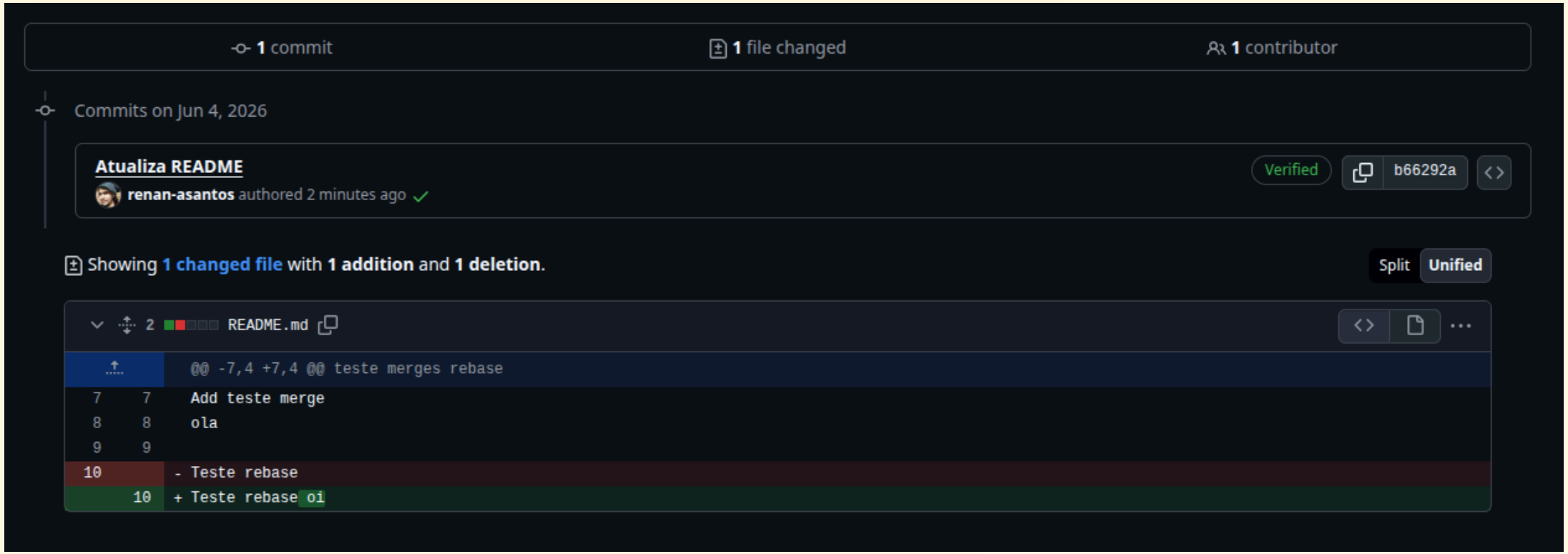
Gitlab possui um nome que eu prefiro: **MRs ou Merge-Requests**



O que são MRs?

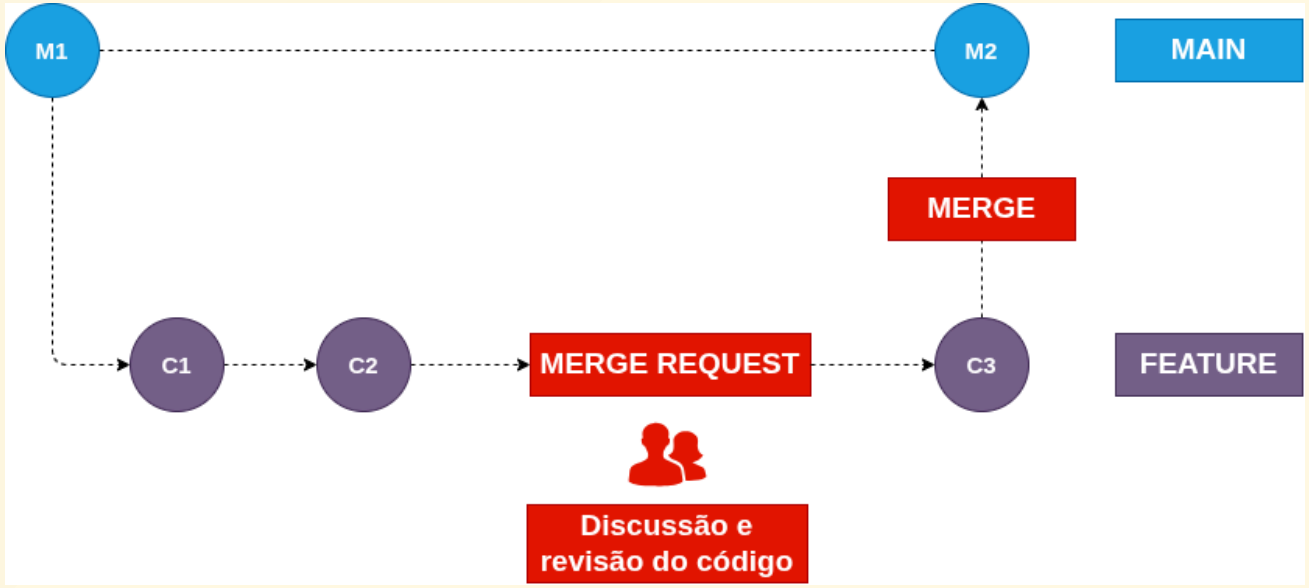

Comparing changes
Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.
base: main
compare: feature-nova
Able to merge. These branches can be automatically merged.
Add a title *
Atualiza README
Add a description
Write
Preview
Uma curta descrição
Markdown is supported
Paste, drop, or click to add files
Create pull request
Reviewers
No reviews
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Use Closing keywords in the description to automatically close issues
Helpful resources

O que são MRs?


1 commit
1 file changed
1 contributor
Commits on Jun 4, 2026
Atualiza README
renan-asantos authored 2 minutes ago
Verified
b66292a
Showing 1 changed file with 1 addition and 1 deletion.
Split
Unified
2
README.md
@@ -7,4 +7,4 @@ teste merges rebase
7 7 Add teste merge
8 8 ola
9 9
10 - Teste rebase
10 + Teste rebase oi

O que são Code Reviews?

“ “ Revisão de código é o processo no qual o autor de uma MR disponibiliza seu código para pelo menos uma outra pessoa avaliar, essa podendo adicionar comentários para o entendimento, estabelencendo-se um diálogo[1] ” ”



1. Definição própria

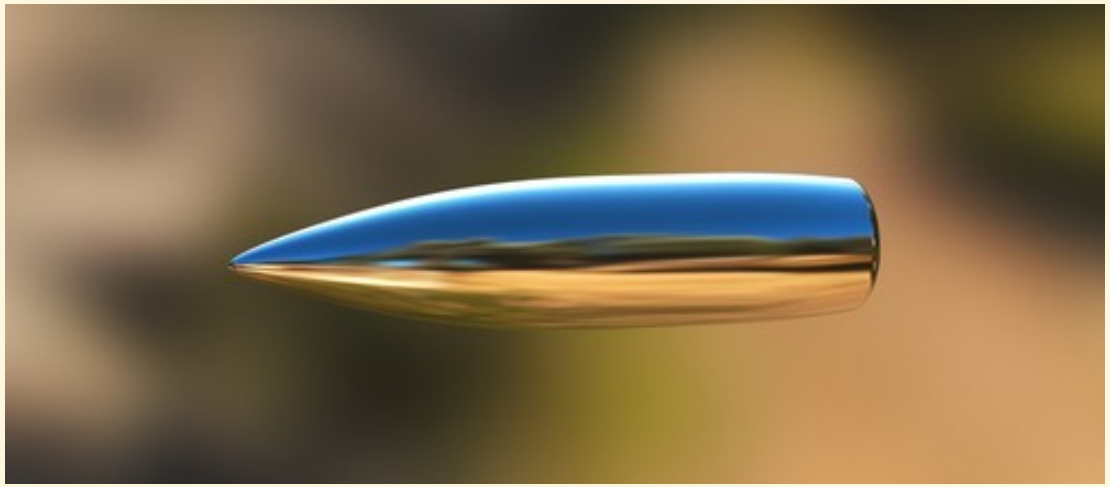
O que TAMBÉM É Code Review?

- Pair Programming, só que assíncrono
- Parte da tarefa
- Última barreira humana pré produção
- Acordo entre o time/ área/empresa



O que NÃO É Code Review?

- Tarefa do time de QA
- "Bala de prata"



Benefícios do Code Review

Benefícios do Code Review

- Além de identificar, previne possíveis bugs
- Aumenta a qualidade do software escrito
- Diminui o risco de dívidas técnicas
- **Oportunidade de aprendizado (para ambos)**

Benefícios do Code Review

Uma boa cultura de code review pode indicar falhas no seu processo...

- Complexidade da tarefa
- Entendimento do que era preciso
- Necessidade



Code Review na era da IA

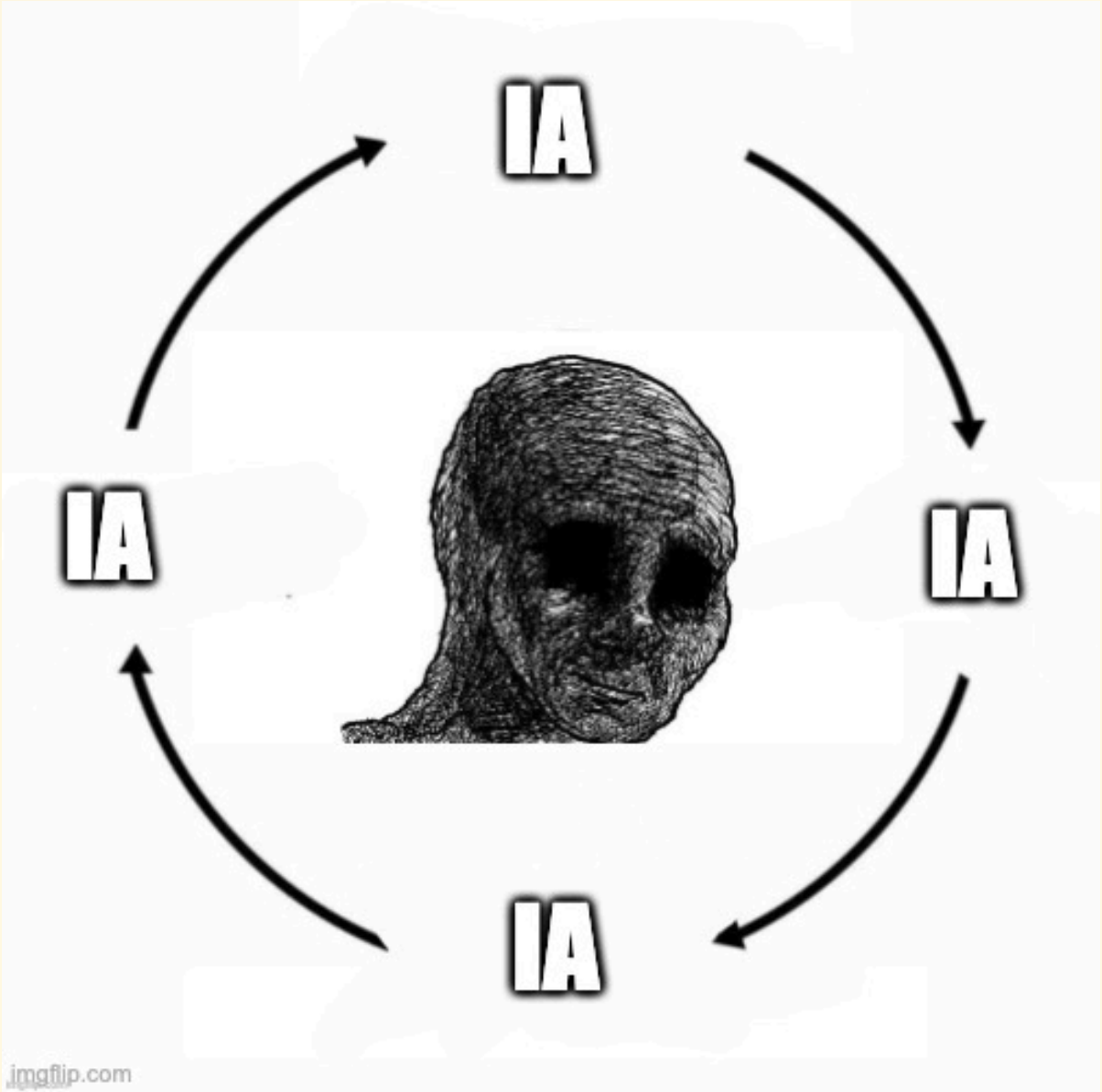
Dado que a IA existe...

- Volume de código gerado é muito maior
- O "pair programming assíncrono" agora é síncrono com humano + IA
- Escrever código era mais lento que revisá-lo. A IA inverteu essa lógica[1]

1. The Final Bottleneck

“ E se fizermos um agente para revisar MRs? ”

- Se a IA escreve o código e a IA também revisa, quem é o responsável?
- Com quem está o conhecimento do código?



Qual a importância do Code Review agora?

- Autores de MRs cada vez menos revisando a saída das IAs
- O review humano vira uma validação de intenção mais do que correção de bugs
- "O time entende o código gerado?” passa a ser critério de DoD
- Mudança do conceito do limite de WIP



Dicas para autores de MRs



Dicas para autores de MRs

- MRs pequenas
- Sem juntar **várias** mini melhorias



Dicas para autores de MRs

O primeiro revisor é sempre **você**!

- CI
- Conflitos
- Testes unitários
- Linter/formatter
- Subiu em dev?



Dicas para autores de MRs

O primeiro revisor é sempre **você**!

IA: Não peça para alguém revisar o que nem você mesmo revisou[1]

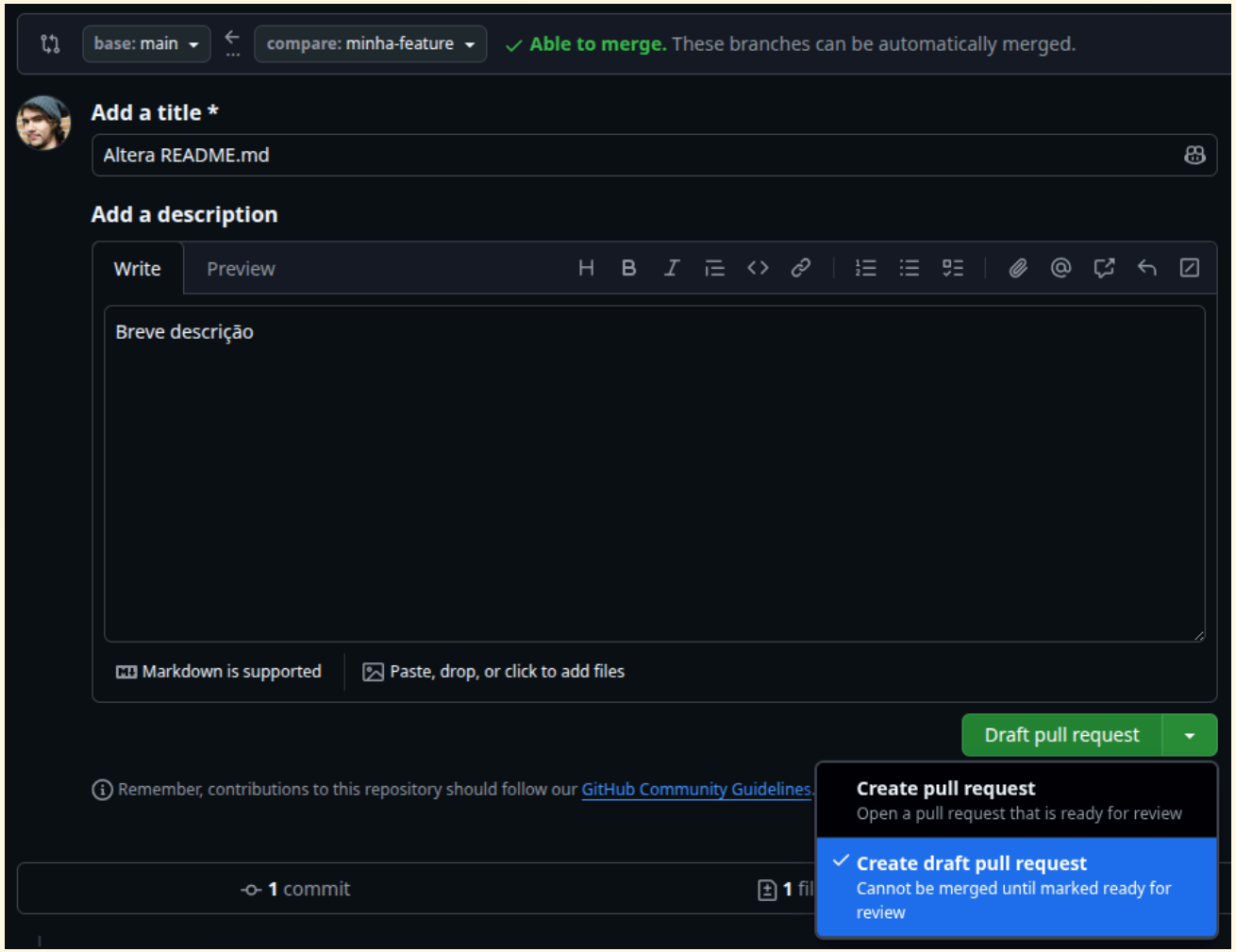


1. Guidelines for Respectful Use of AI

Dicas para autores de MRs

Crie uma MR em draft

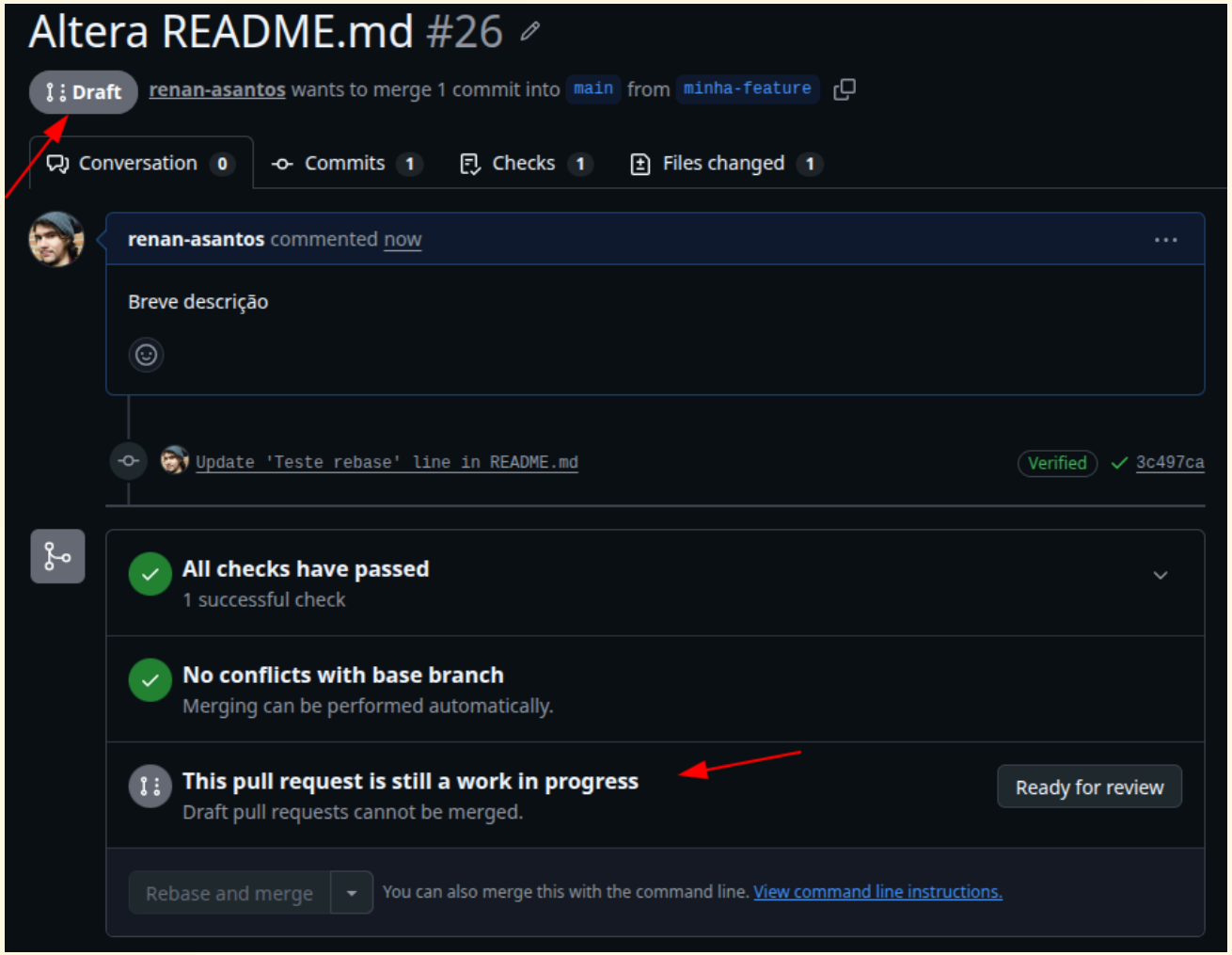
- opinião rápida sobre a abordagem
- “erre rápido” para corrigir rápido



Dicas para autores de MRs

Crie uma MR em draft

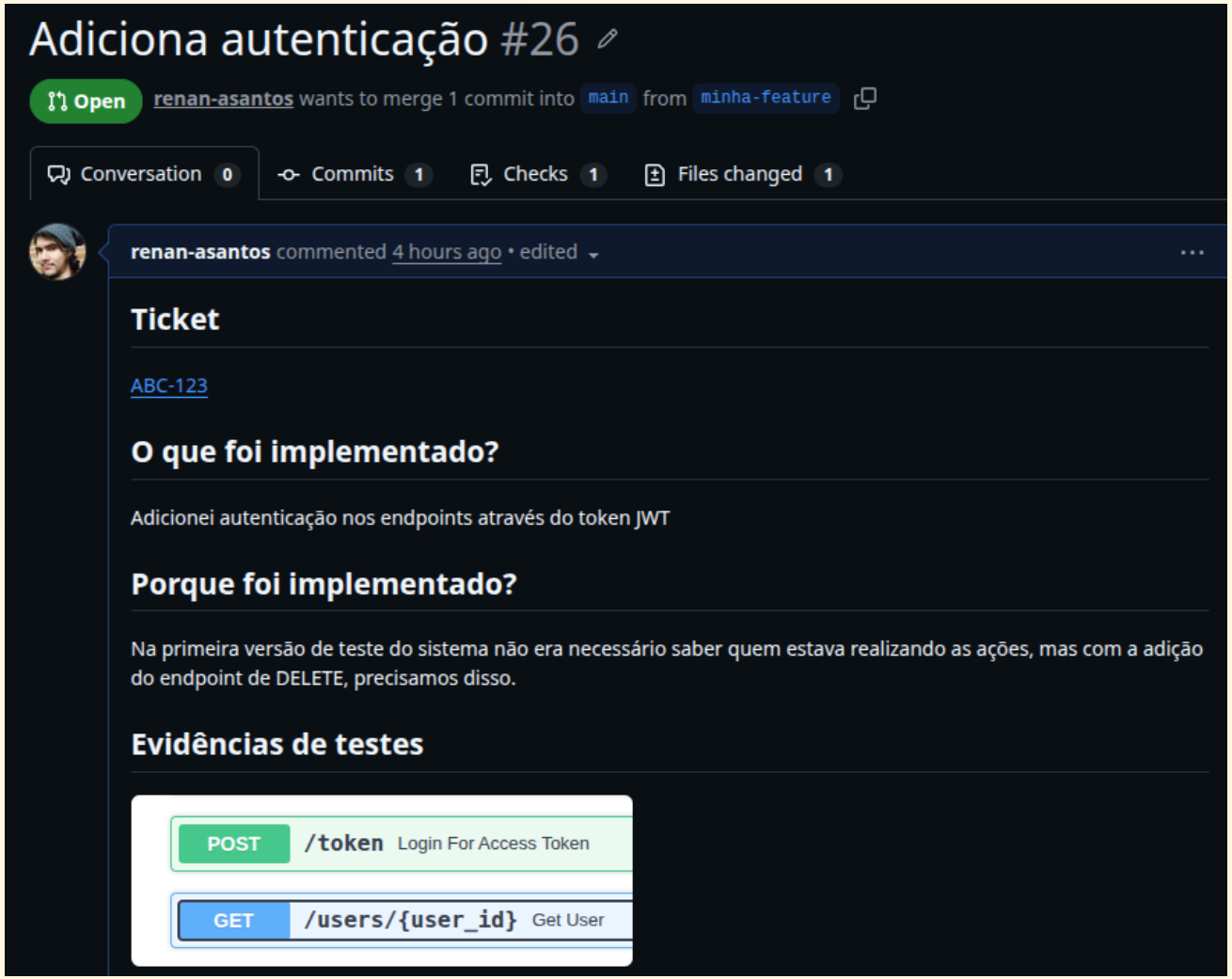
- opinião rápida sobre a abordagem
- “erre rápido” para corrigir rápido



Dicas para autores de MRs

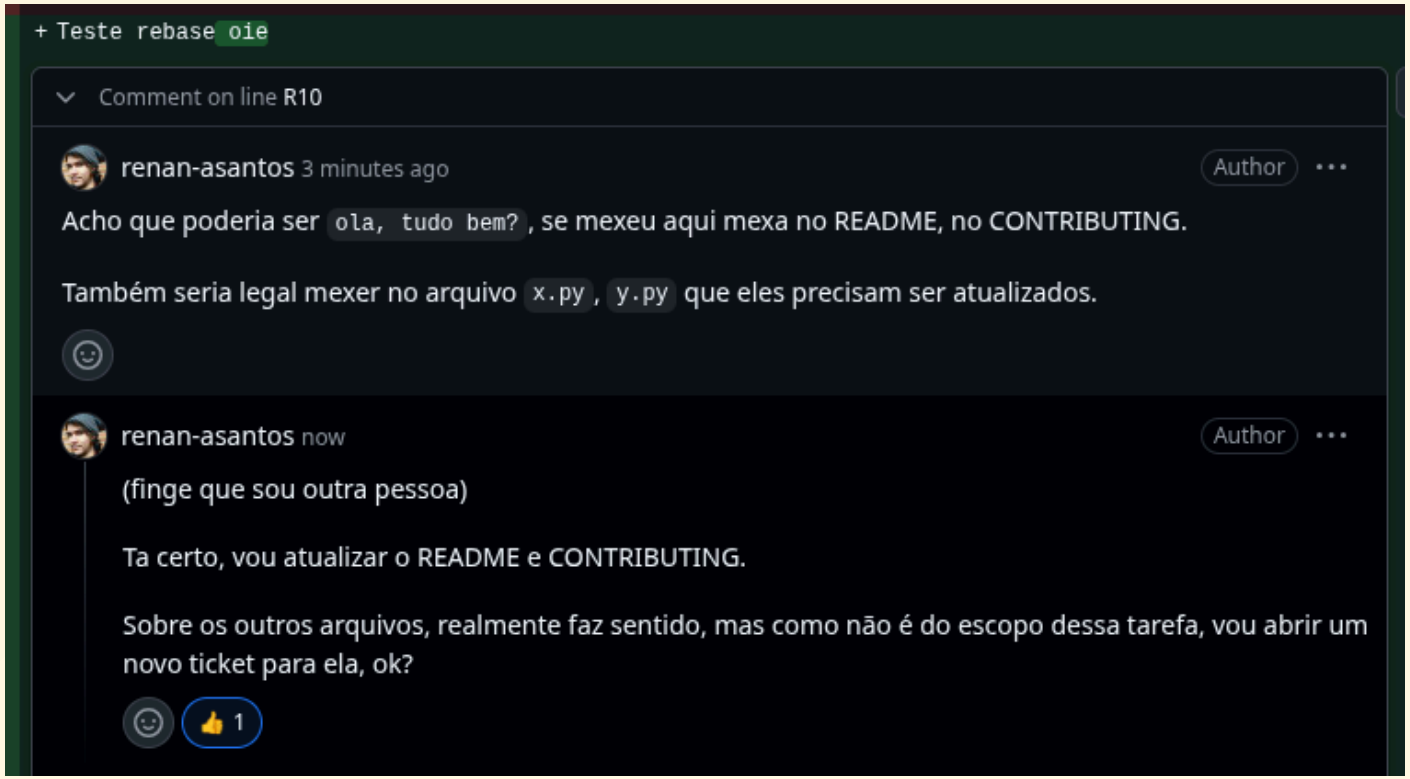
Forneça **contexto** para quem vai ler a MR

- Escreva uma descrição clara e bem exemplificada
- Roteiro de revisão
- Adicione imagens, gifs, vídeos



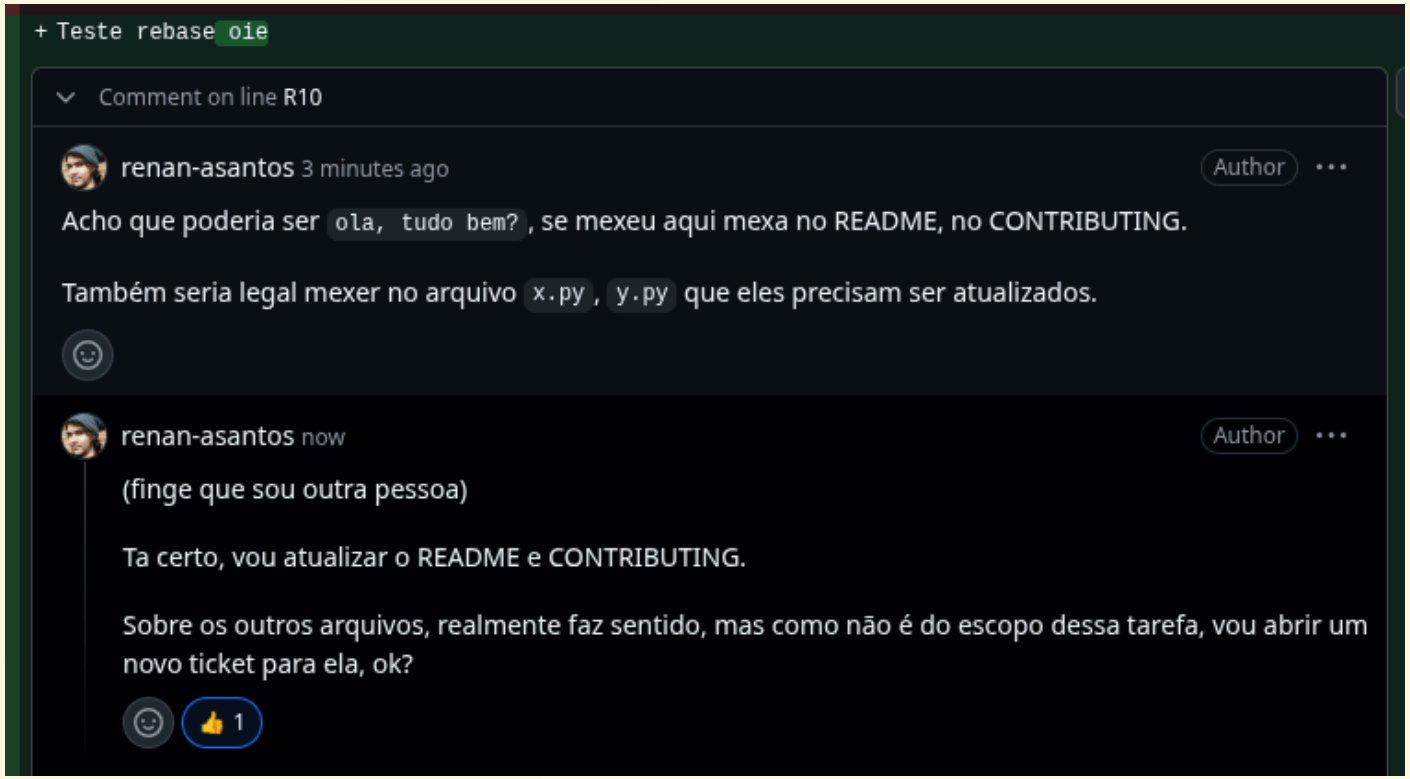
Dicas para autores de MRs

- Discutir nos comentários, não no pv
- Responda o quanto antes os comentários
- Muitas sugestões de melhoria? Abra outro ticket



Dicas para autores de MRs

- Não leve para o pessoal, você não é seu código
- **Seja gentil e se coloque no lugar do revisor**

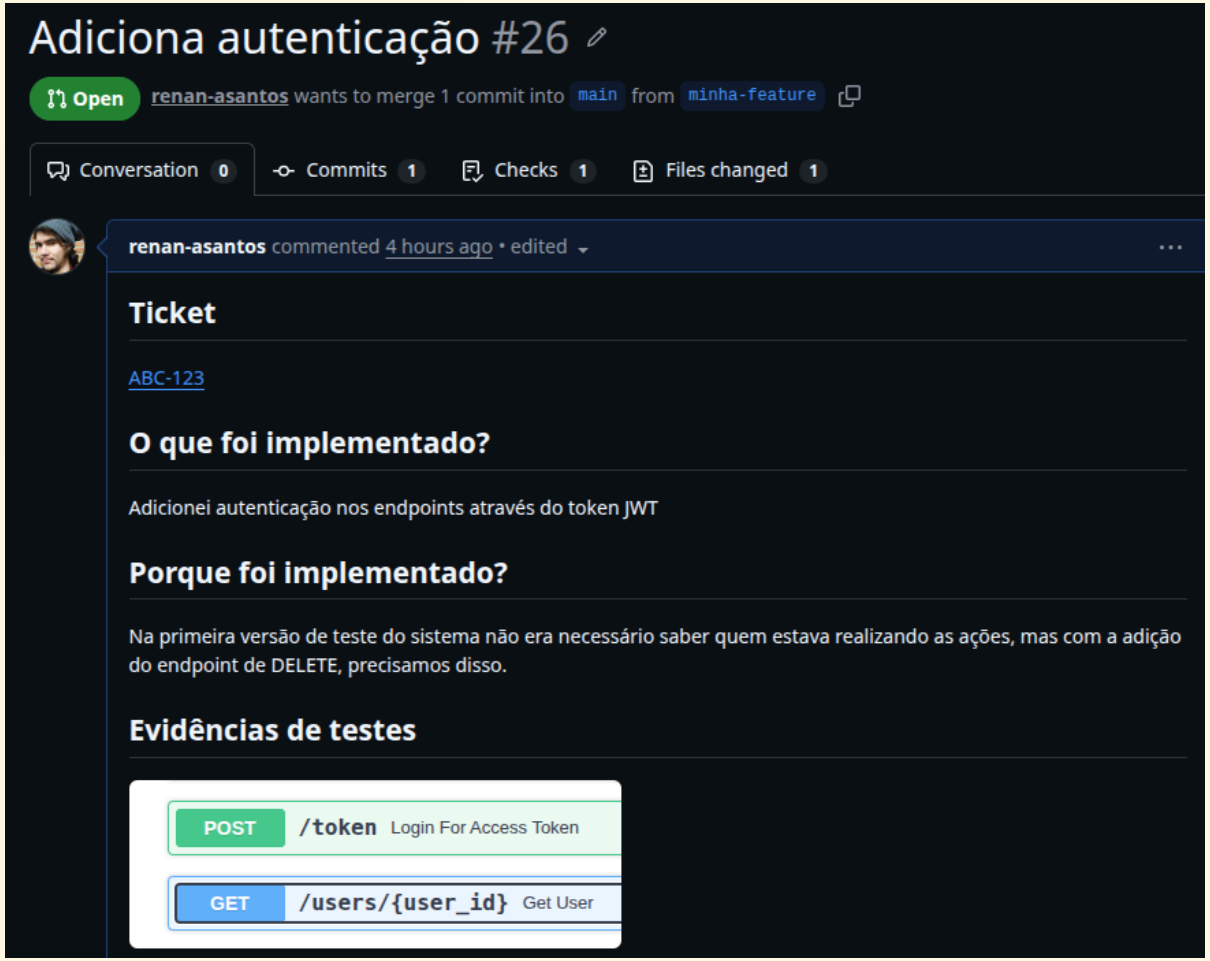


Dicas para revisores de MRs

Dicas para revisores de MRs

Adquira contexto antes de revisar

- O autor caprichou na descrição? Então use-a!
- Leia o card atrelado à tarefa



Dicas para revisores de MRs

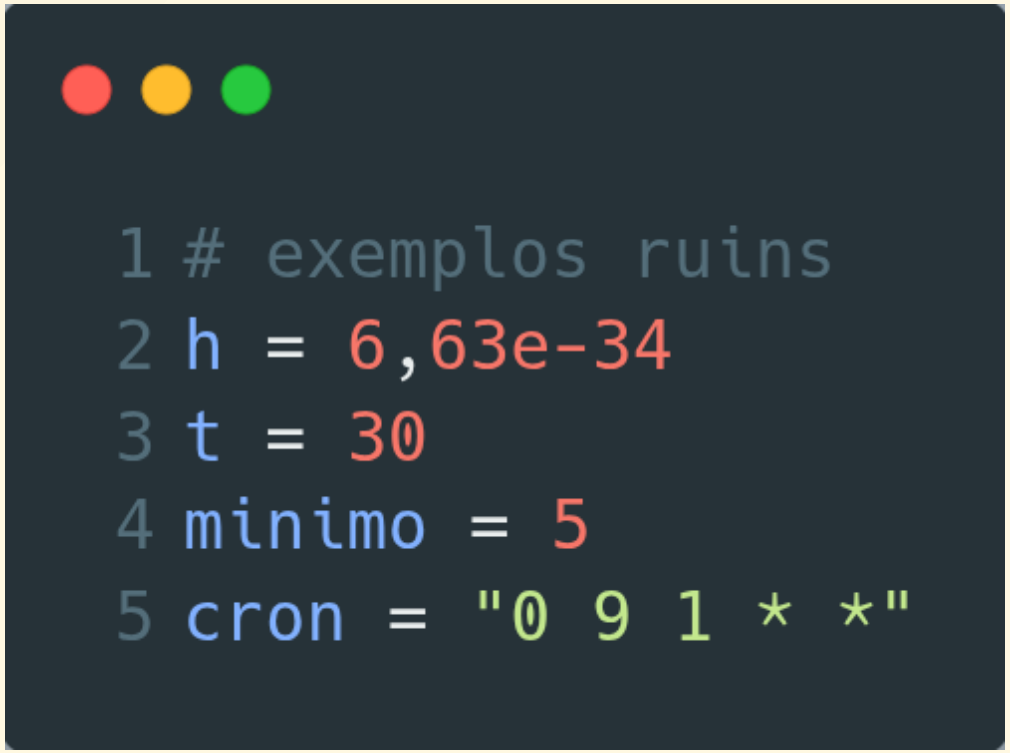
Roteiro pessoal

- Começe pelos arquivos não de código fonte, adições de libs, arquivos de configurações, documentação
- Alterne entre o código fonte e os testes
- Siga o fluxo desde o começo, por exemplo
  `endpoint` -> `service` -> `utils`

Dicas para revisores de MRs

Nos arquivos de código...

- Analise nomes de variáveis, estão claros e intuitivos?
- **IA**: agente de revisor de MR pode ajudar



Dicas para revisores de MRs

Nos arquivos de código...

```
1 # exemplos ruins
2 h = 6,63e-34
3 t = 30
4 minimo = 5
5 cron = "0 9 1 * *"
```

```
1 # exemplos bons
2 constante_de_planck = 6,63e-34
3 tempo_limite_segundos = 30
4 minimo_itens_carrinho = 5
5 cron_todo_dia_1_9h = "0 9 1 * *"
```

Dicas para revisores de MRs

Nos arquivos de código...

- Existe over-engineering?
- Quem ler entenderá a solução no futuro?

```python
class Saudacao:
  def __init__(self,nome):
    self.nome = nome
    self._configurar()
  def _configurar(self):
    self._template = "Olá, {self.nome}!"
  def dizer(self):
    return self._template.format(self=self)

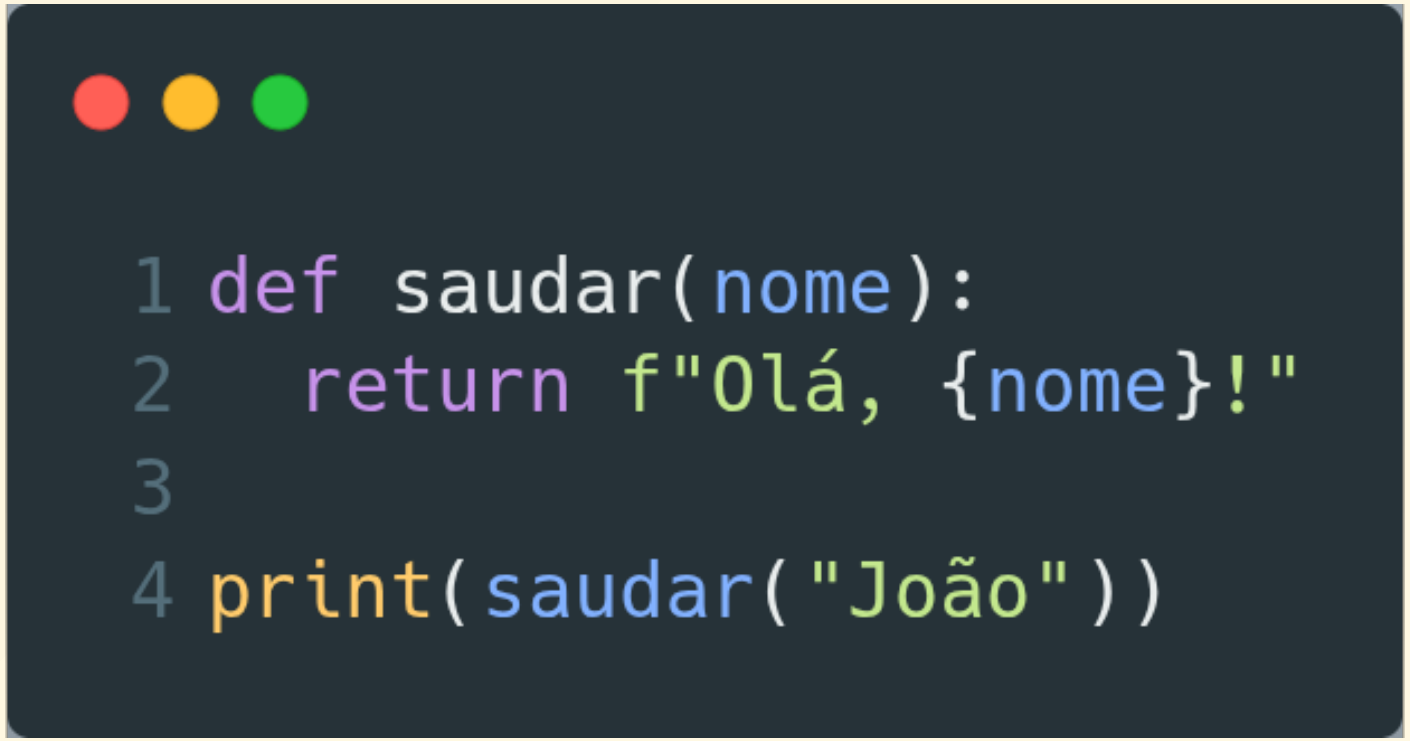
print(Saudacao("João").dizer())
```

Dicas para revisores de MRs

Lembre do Zen do Python:

“ “ Simple is better than complex ” ”

IA: tende a deixar mais complexo, então cuidado[1]



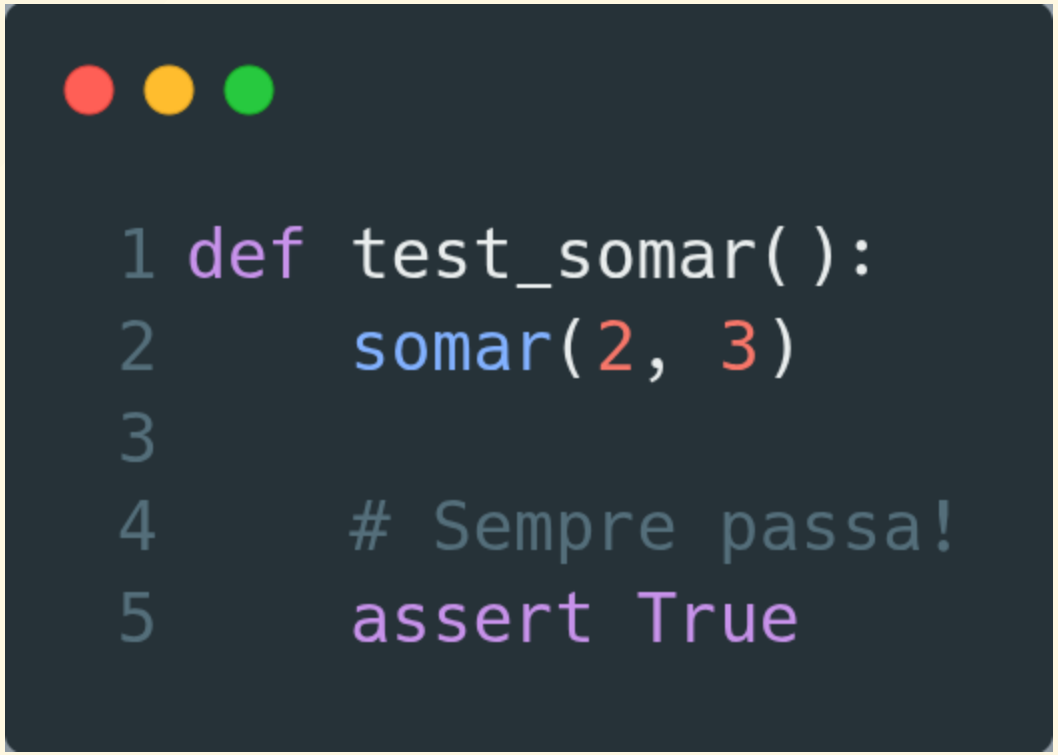
```
1 def saudar(nome):
2     return f"Olá, {nome}!"
3
4 print(saudar("João"))
```

1. Guidelines for Respectful Use of AI

Dicas para revisores de MRs

Nos arquivos de testes...

- Estão corretos e bem arquitetados?
- Bons nomes?



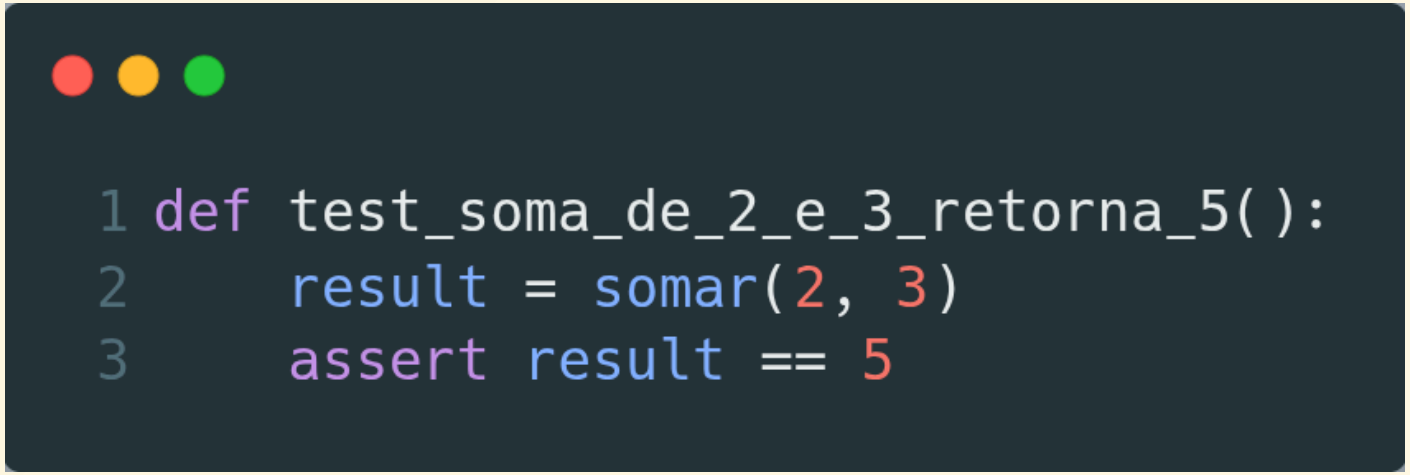
```python
def test_somar():
    somar(2, 3)

    # Sempre passa!
    assert True
```

Dicas para revisores de MRs

Nos arquivos de testes...

- Estão corretos e bem arquitetados?
- Bons nomes?



```python
def test_soma_de_2_e_3_retorna_5():
    result = somar(2, 3)
    assert result == 5
```

Dicas para revisores de MRs

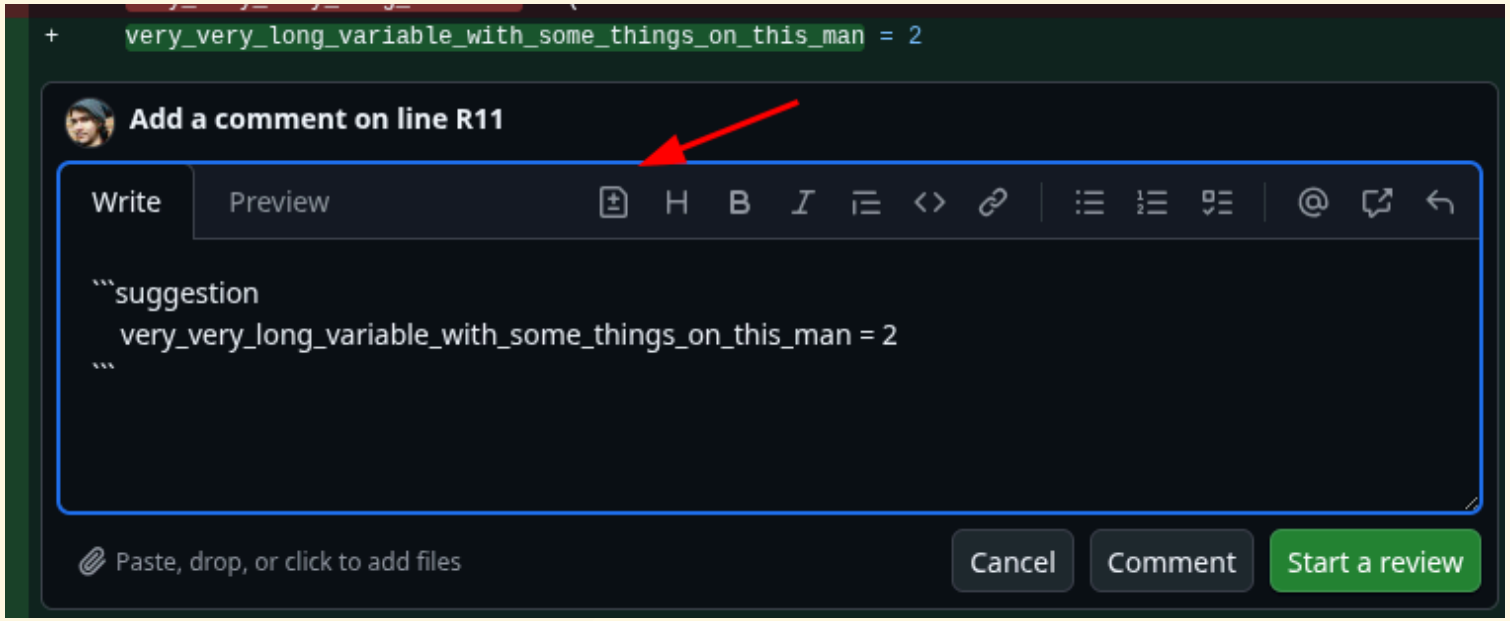
Nas MRs mais críticas, faça checkout da branch, rode você mesmo e valide



Dicas para revisores de MRs

Ao comentar na MR...

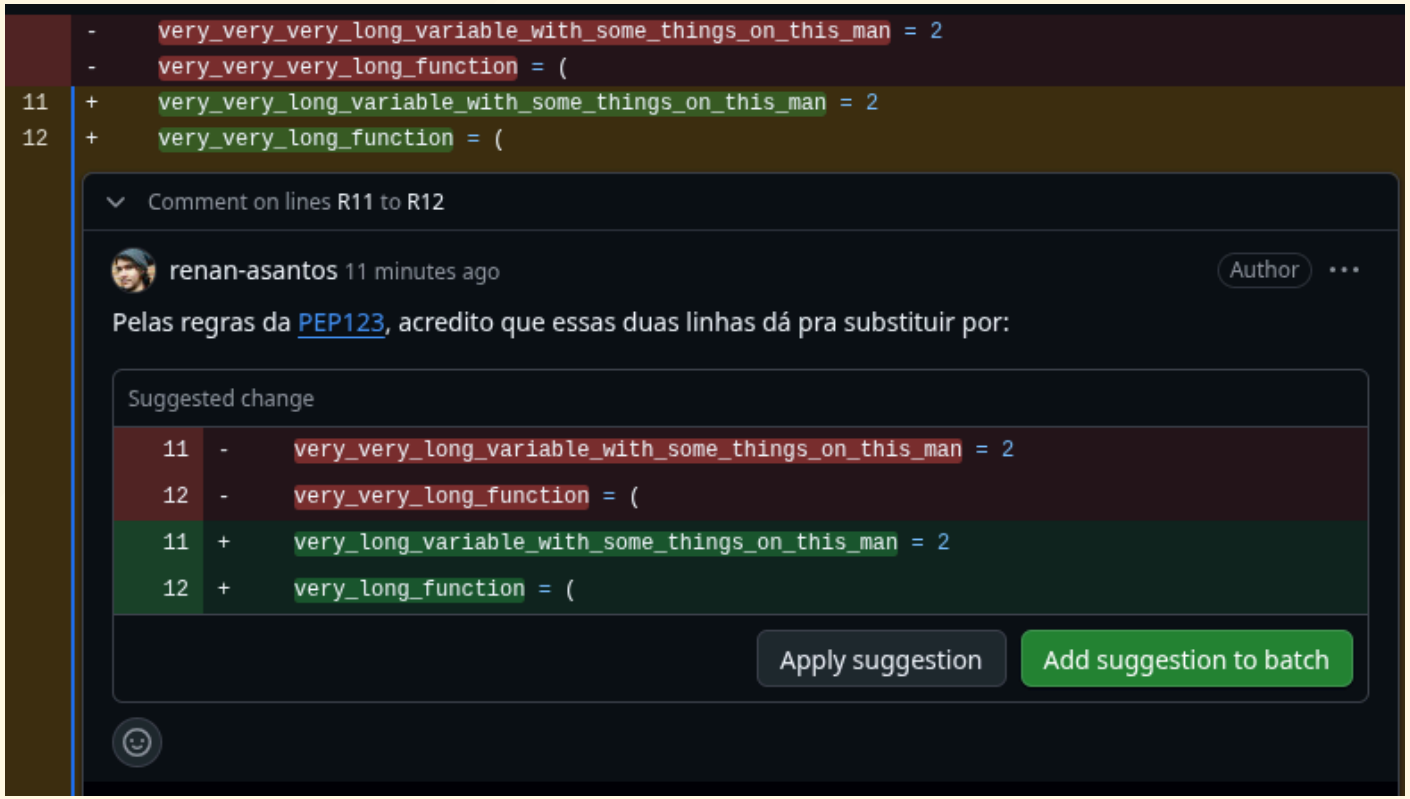
- Adicione sugestões de código



Dicas para revisores de MRs

Ao comentar na MR...

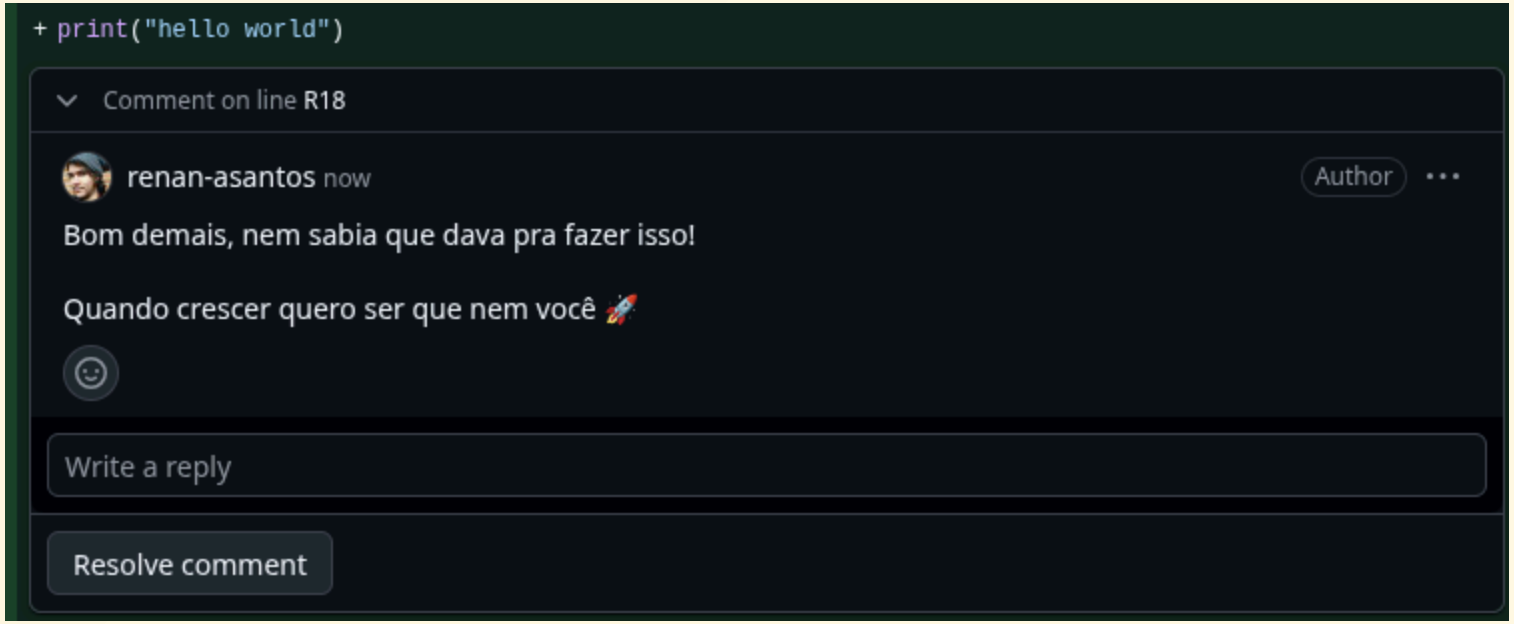
- Selecione trechos de código específicos
- Explique sua sugestão, coloque referências, exemplos, imagens, etc



Dicas para revisores de MRs

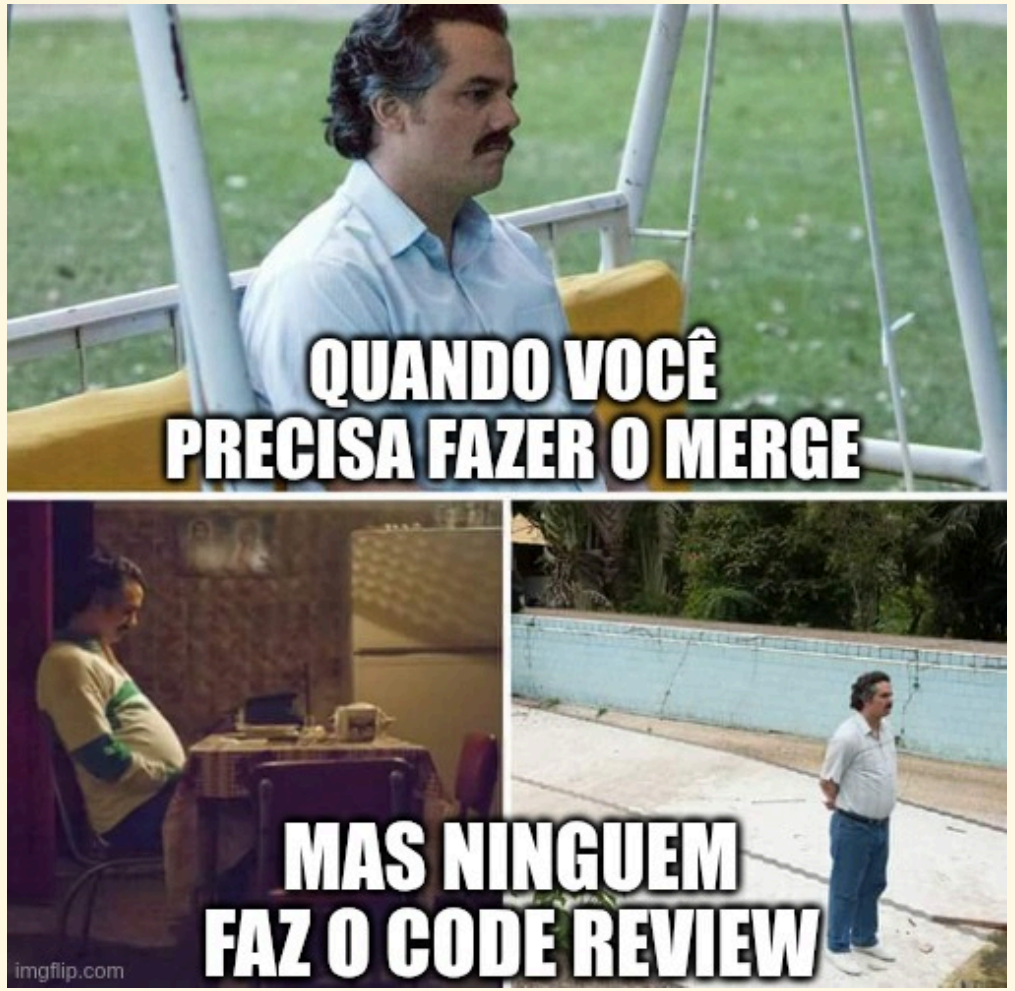
Ao comentar na MR...

- **Enalteça** pontos interessantes, elogie códigos bons e comente se aprendeu algo novo



Dicas para revisores de MRs

- Se você não está revisando a MR de alguém, essa pessoa está travada.
- **IA**: Dilema do revisor (“Cognitive Debt”)



Dicas para revisores de MRs

- Cuidado com o efeito manada
- Revisem, nem que seu review não "conte"



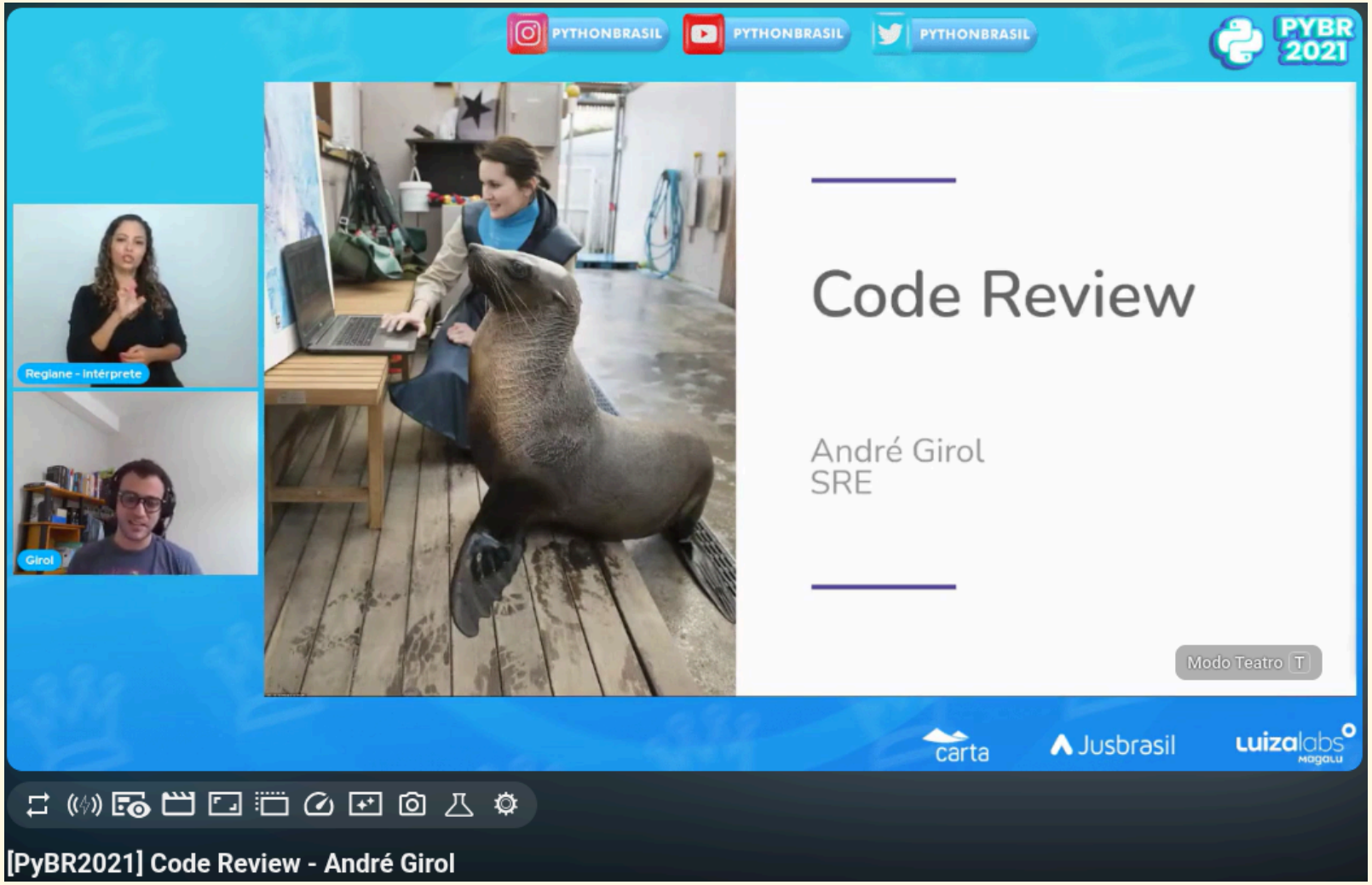
O que quero que levem com vocês

O que quero que levem com vocês

- Code review é um pair programming assíncrono ("diálogo")
- Parte da **sua** tarefa e dono da MR
- Não é uma "bala de prata"
- Dado que a IA existe, review mais importante
- **Oportunidade de aprendizado**

Agradecimentos

Palestra do [André Girol de Code Review na Python Brasil 2021]



Agradecimentos

Post no blog do Osvaldo Santana - Agilidade em tempos de IA



Dúvidas?

Linkedin: /in/renan-asantos/
Telegram: @renan_asantos



Slides no QRCode